# Code-Layout, Readability and Reusability

| | |
|---|---|
| **Date** | 10 July 2026 |
| **Team ID** | XXXXXX |
| **Project Name** | Raising Waters |
| **Maximum Marks** | 5 Marks |

## Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

## Code Layout Checklist:

| S.No | Code Quality Parameter | Description | Followed (Yes / No / Partial) | Remarks |
|---|---|---|---|---|
| 1 | Consistent Indentation | Uniform spacing/tabs used throughout the code | Yes | Follows PEP 8 standards using 4 spaces per indentation level throughout all Python files. |
| 2 | Proper File Structure | Files and folders are logically organized | Yes | Separate directories exist for static UI files, templates, models (.pkl), and core data utilities. |
| 3 | Meaningful Variable Names | Variables reflect their purpose clearly | Yes | Names explicitly describe data features ( e.g., rainfall_metrics, risk_tier_output |
| 4 | Function / Method Names | Functions are descriptively named | Yes | Explicit naming conventions used, such as load_ml_model() and evaluate_flood_risk() |
| 5 | Code Comments | Inline and block comments explain log c | Yes | Every major data processing block and model inference step contains concise documentation comments. |
| 6 | Modular Design | Code is split into reusable functions/m dules | Yes | Data ingestion, model inference, and web rendering routing are decoupled into separate components |
| 7 | No Redundant Code | Duplicate or unused code is removed | Yes | Leftover experimentation code and unused library imports have been refactored out. |
| 8 | Error Handling | Exceptions and errors are handled gracefully | Yes | Try-except blocks handle external API network timeouts and data validation issues smoothly. |

## Reusable Components / Modules:

| S.No | Component / Module Name | Language / Technology | Where Reused | Reusability Level (High / Medium / Low) |
|---|---|---|---|---|
| 1 | weather_api_fetcher | Python / Requests | Triggered both during real-time dashboard updates and manual database retraining processes. | High |

| | | | | |
|---|---|---|---|---|
| 2 | ml_inference_engine | Python / Requests | Triggered both during real-time dashboard updates and manual database retraining processes. | High |
| 3 | alert_logger | Python / CSV Utility | Utilized by any module that needs to record data validation errors or successful prediction runs. | Medium |
| 4 | | | | |
| 5 | | | | |

## Overall Code Quality Assessment:

| Aspect | Rating (1-5) | Comments |
|---|---|---|
| Code Layout & Structure | 5 | Clean structure that follows standard directory setups for machine learning web apps. |
| Readability | 4 | Highly readable layout with descriptive naming styles used consistently. |
| Reusability | 4 | Utility functions are isolated nicely, enabling rapid updates to model components. |
| Documentation / Comments | 5 | Clear docstrings and inline comments detailing function inputs and outputs. |
| Overall Score | 4.5 | Well-architected codebase designed for easy maintainability. |